How the Altradits Lightning node was built

A step-by-step record of the LND node on this Mac, with the pruning question explained. Written 8 November 2026 from the setup sessions of 22 and 23 October 2026.

The node at a glance

Part	What it is
Software	LND 0.21.3-beta, compiled from the LND source code (checkout at ~/lnd, tag v0.21.3-beta). Programs: ~/bin/lnd and ~/bin/lncli.
Network	Bitcoin mainnet (real money)
Blockchain source	**Neutrino**, a light client. No Bitcoin Core, no full blockchain and no pruning (see next section).
Data folder	~/.lnd-altradits (about 490 MB today)
Name / ID	Altradits, [your node's public key]
Reachable by	Lightning peers over Tor (.onion address). The altradits.com website over an ngrok tunnel.
Channel	1 private channel from Magma's seller, 2,943,434 sats. Today about 98,000 sats are on your side and 2.84M sats can still be received.
Starts by	macOS launch agents at login (added 8 November 2026)

Pruning: the short answer

> **This node does not prune, because it never stores the blockchain at all.** It uses Neutrino, which is lighter than pruning. Pruning is a Bitcoin Core setting. The only Bitcoin Core on this Mac is the regtest one from your bootcamp (rpcuser bootcamp, port 18443, about 37 MB), and it has nothing to do with the Lightning node.

Pruning came up in your bitcoin-core-lab course (module 11 runs Bitcoin Core with `-prune=5500`), which is probably why it sounds familiar. Here is how it works, and why Neutrino was used instead.

What pruning does, step by step

1. **Download every block.** A pruned Bitcoin Core node still downloads the whole blockchain from 2009 to today, more than 700 GB.

2. **Check every block.** It verifies every transaction and signature itself, exactly like a full node. Pruning saves disk space, not checking.

3. **Keep the summary.** While checking, it builds the *UTXO set*, the list of every coin that can still be spent. This is all a node needs to judge new transactions (about 10 GB).

4. **Delete old blocks.** Once a block file is checked and the UTXO set updated, it deletes the oldest block files, keeping only the most recent ones up to the prune size. `prune=550` keeps 550 MB (the minimum); `prune=5500` keeps about 5.5 GB.

5. **Carry on.** New blocks are checked and added, and the oldest are deleted, so disk use stays flat.

The cost is the first sync: downloading and checking 700+ GB takes days, and the Mac must stay on and online the whole time. A pruned node also can't serve old blocks to others or rescan an old wallet without downloading everything again.

What Neutrino does instead

1. **Download block headers only.** Each block's 80-byte header, about 78 MB for the whole chain. LND checks the proof of work on every header, so it follows the real chain.

2. **Download block filters.** A small fingerprint per block (BIP 157/158) that says which addresses a block might touch.

3. **Fetch only the blocks it needs.** When a filter matches your wallet or channel, LND asks a peer for that one block and checks it.

This node synced in under an hour on 22 October and uses about 490 MB.

The three options side by side

	Full node	Pruned node	Neutrino (this node)
Disk	700+ GB	About 10 GB plus prune size	Under 1 GB
First sync	Days	Days (same download)	Minutes to an hour
Checks every transaction	Yes	Yes	No. Checks headers, and only the blocks it uses
Depends on others	No	No	Yes: peers must serve filters (btcd-mainnet.lightning.computer)
Privacy	Best	Best	Good. Peers see which blocks you fetch
Fits this Mac	No: 44 GB free of 228 GB	Tight, and days of downloading	Yes

Why Neutrino: a full node can't fit on this Mac, and a pruned one would need days of downloading before the shop could take a payment. For a node that only receives shop payments, Neutrino was the practical choice. The trade-off is that it trusts its filter peer to be available and honest about filters, while the proof of work on every header is still checked locally.

If you want a pruned node later

You can move LND from Neutrino to your own pruned Bitcoin Core without losing the wallet or channel. In outline (don't run this without planning it first):

```
# 1. Mainnet bitcoin.conf (separate from the regtest bootcamp one)
prune=5500
server=1
zmqpubrawblock=tcp://127.0.0.1:28332
zmqpubrawtx=tcp://127.0.0.1:28333

# 2. Let Bitcoin Core finish its first sync (days), then in lnd.conf:
bitcoin.node=bitcoind
bitcoind.rpchost=127.0.0.1:8332
bitcoind.zmqpubrawblock=tcp://127.0.0.1:28332
bitcoind.zmqpubrawtx=tcp://127.0.0.1:28333
```

LND works with a pruned Bitcoin Core: if it needs an old block that was deleted, it fetches it from other peers. You'd need about 15 GB free and several days for the first sync.

How it was built, step by step

Commands are shown as they were run. Secrets (wallet password, 24-word seed, ngrok token, website key) are never shown here and must never be shared.

Step 0. Get LND (before 22 Oct)

LND 0.21.3-beta was compiled from its source code (Lightning Labs' GitHub, tag v0.21.3-beta, in ~/lnd) and the programs were placed in ~/bin.

```
~/bin/lnd --version     # lnd version 0.21.3-beta
```

Step 1. Write the config file (22 Oct)

A private folder and `lnd.conf`. Everything listens only on this Mac (127.0.0.1), and `bitcoin.node=neutrino` is the line that chooses Neutrino over Bitcoin Core.

```
mkdir -p ~/.lnd-altradits && chmod 700 ~/.lnd-altradits

[Application Options]
alias=Altradits
color=#F7931A
listen=127.0.0.1:9735          # Lightning peers
rpclisten=127.0.0.1:10009      # lncli
restlisten=127.0.0.1:8080      # website, via ngrok
accept-keysend=false

[Bitcoin]
bitcoin.mainnet=true
bitcoin.node=neutrino

[neutrino]
neutrino.addpeer=btcd-mainnet.lightning.computer

[fee]
fee.url=https://nodes.lightning.computer/fees/v1/btc-fee-estimates.json

[protocol]
protocol.option-scid-alias=true
protocol.zero-conf=true
```

Two other peers (zaphq.io) were in the first version but no longer exist, so they were removed.

Step 2. Start the node and sync headers (22 Oct)

```
~/bin/lnd --lnddir=$HOME/.lnd-altradits
```

It began downloading headers at about 12,000 blocks every 10 seconds, then waited for a wallet.

Step 3. Create the wallet and the 24-word seed (22 Oct)

```
~/bin/lncli --lnddir=$HOME/.lnd-altradits --network=mainnet create
```

A wallet password typed twice, **n** for a new seed, no seed passphrase. The 24 words were written on paper, checked letter by letter, and the screen cleared. The seed plus `channel.backup` is how funds are recovered if the Mac is lost.

By 23:34 the node was live: synced to block 972,672 with 3 peers.

Step 4. Unlock automatically after a restart (22 Oct)

The wallet password went into a file only your Mac user can read, and lnd.conf points to it:

```
wallet-unlock-password-file=/Users/you/.lnd-altradits/wallet-password    # file is -rw-------
```

Trade-off: anyone who gets into your Mac account could read that file.

Step 5. Open a tunnel for the website (22 to 23 Oct)

altradits.com runs on HostAfrica and can't reach a Mac at home directly. A free ngrok account gives the node a fixed public address, [your-domain].ngrok-free.dev.

```
ngrok config add-authtoken <token>      # once
ngrok http https://localhost:8080 --url=https://[your-domain].ngrok-free.dev
```

Step 6. Connect the website (23 Oct)

In DirectAdmin's Node.js settings the site got `LND_REST_URL` (the ngrok address) and `LND_INVOICE_MACAROON`, a receive-only key that can create invoices but never spend. PR #4 added the route hints that private channels need.

Step 7. Give the node a Tor address (23 Oct)

```
brew install tor && brew services start tor
printf '%s\n' 'SOCKSPort 9050' 'ControlPort 9051' 'CookieAuthentication 1' \
  > /opt/homebrew/etc/tor/torrc && brew services restart tor

# added to lnd.conf
[tor]
tor.active=true
tor.v3=true
tor.skip-proxy-for-clearnet-targets=true
tor.control=localhost:9051
```

The node now has a permanent .onion address, so sellers can reach it without opening a port on your router. Normal internet peers stay direct, so syncing stays fast.

Step 8. Buy inbound liquidity (23 Oct)

A node can't receive until someone opens a channel to it with their own funds. A $10 (12,244 sats) anonymous order on Amboss Magma, paid from Wallet of Satoshi, asked for a private channel from reliable sellers only (order [order ID]). Magma's node checked yours over Tor within a minute, and the seller opened a 2,943,434-sat channel.

Step 9. Lock it down (23 to 25 Oct)

ngrok policy: only the website server ([website server IP]) can reach the node through the tunnel.

```
# ~/.lnd-altradits/ngrok-policy.yml
on_tcp_connect:
  - actions:
      - type: restrict-ips
        config:
          enforce: true
          allow:
            - [website server IP]/32
```

Keys: the full-control admin key is deleted after each start. Daily checks use the read-only key (alias `lncli-ro`). The website key was confirmed receive-only.

Step 10. Start automatically (8 Nov)

Two launch agents, com.altradits.lnd and com.altradits.ngrok, start the node and tunnel at login. Tor already starts through Homebrew. See the separate *Lightning Node Auto-Start Guide*.

Checking it today

```
~/bin/lncli --lnddir=$HOME/.lnd-altradits --network=mainnet \
  --macaroonpath=$HOME/.lnd-altradits/data/chain/bitcoin/mainnet/readonly.macaroon \
  getinfo | grep -E 'synced_to_chain|num_active_channels|block_height'
```

On 8 November 2026: synced, block 975,042, 1 active channel.

If the Mac is lost or broken

You need two things: the **24-word seed on paper**, and a recent copy of `~/.lnd-altradits/data/chain/bitcoin/mainnet/channel.backup`. On a new machine, install LND, run `lncli create`, choose **y** to restore from the seed, and give it the channel backup. The channel is then closed and its funds return to your wallet. Keep a copy of channel.backup somewhere other than this Mac, and update it after any channel change.